

Linux Engineer Command Reference

A concise practical reference covering the most important Linux commands for software, AI/ML, DevOps, cloud, and backend engineers.

1. Navigation

Command	Description
<code>pwd</code>	Show current directory
<code>ls</code>	List files and directories
<code>ls -l</code>	Detailed listing
<code>ls -la</code>	Detailed listing including hidden files
<code>cd DIR</code>	Change directory
<code>cd ..</code>	Go to parent directory
<code>cd .</code>	Stay in current directory
<code>cd ~</code>	Go to home directory
<code>cd /</code>	Go to filesystem root
<code>cd -</code>	Go to previous directory

```
pwd
ls -la
cd /home/sheharyar
cd ..
cd ~
```

2. Linux Paths

```
/    → filesystem root
~    → current user's home
.    → current directory
..   → parent directory
```

Absolute Path

/home/sheharyar/project/main.py

Starts from `/`.

Relative Path

project/main.py

Starts from the current directory.

3. Files and Directories

Command	Description
<code>mkdir DIR</code>	Create directory
<code>mkdir -p a/b/c</code>	Create nested directories
<code>touch FILE</code>	Create empty file
<code>cp FILE DEST</code>	Copy file
<code>cp -r DIR DEST</code>	Copy directory
<code>mv OLD NEW</code>	Rename/move
<code>rm FILE</code>	Delete file
<code>rm -r DIR</code>	Delete directory recursively
<code>rmdir DIR</code>	Delete empty directory
<code>rm -i FILE</code>	Ask before deleting
<code>rm -rf DIR</code>	Force recursive deletion

Examples:

```
mkdir project
touch main.py
cp main.py backup.py
mv backup.py backup_main.py
rm backup_main.py
```

4. Viewing Files

Command	Description
<code>cat FILE</code>	Display entire file
<code>less FILE</code>	Read file page by page
<code>head FILE</code>	Show beginning
<code>head -n 20 FILE</code>	Show first 20 lines
<code>tail FILE</code>	Show end
<code>tail -n 20 FILE</code>	Show last 20 lines
<code>tail -f FILE</code>	Follow a changing file
<code>nl FILE</code>	Show line numbers

Examples:

```
cat README.md
less README.md
head -n 10 app.log
tail -f app.log
```

Press `q` to exit `less`.

5. Searching Files and Text

find

Search for files/directories.

```
find . -name "main.py"
```

Search by type:

```
find . -type f
```

```
find . -type d
```

Search by extension:

```
find . -name "*.py"
```

grep

Search text inside files.

```
grep "error" app.log
```

Useful options:

```
grep -i "error" app.log
```

```
grep -r "TODO" .
```

```
grep -n "error" app.log
```

-i → ignore case

-r → recursive

-n → show line number

6. File Information

Command	Description
---------	-------------

<code>file</code>	Identify file type
-------------------	--------------------

<code>FILE</code>	
-------------------	--

<code>stat</code>	Detailed file metadata
<code>FILE</code>	
<code>wc FILE</code>	Count lines/words/bytes
<code>du -h</code>	Disk usage
<code>df -h</code>	Filesystem disk space

Examples:

```
file model.pkl
stat model.pkl
du -sh project/
df -h
```

7. Permissions

Linux permissions are commonly represented as:

r → read
w → write
x → execute

Example:

`-rwxr-xr--`

Change permissions:

```
chmod 755 script.sh
chmod +x script.sh
chmod -x script.sh
```

Common permissions:

755 → `rwxr-xr-x`
644 → `rw-r--r--`
600 → `rw-----`

8. Ownership

Check ownership:

```
ls -l
```

Change owner:

```
sudo chown user file.txt
```

Change owner and group:

```
sudo chown user:group file.txt
```

Change directory recursively:

```
sudo chown -R user:group project/
```

Change group:

```
sudo chgrp group file.txt
```

9. Users

Command	Description
<code>whoami</code>	Current username
<code>id</code>	UID, GID and groups
<code>groups</code>	Current user's groups
<code>who</code>	Logged-in users
<code>w</code>	Logged-in users + activity
<code>adduser USER</code>	Create user
<code>deluser USER</code>	Delete user

`passwd` Change password
`USER`

`usermod` Modify user

Examples:

`whoami`
`id`
`groups`
`sudo adduser ali`
`sudo passwd ali`

10. Root and Sudo

`root` is the Linux superuser.

`sudo` command

Run a command with elevated privileges.

`sudo apt update`
`sudo systemctl restart ssh`
`sudo whoami`

Become root temporarily:

`sudo -i`

Leave root:

`exit`

Switch user:

`su - username`

11. Processes

A process is a running program.

Command	Description
<code>ps</code>	Show processes
<code>ps aux</code>	Detailed process list
<code>top</code>	Real-time process monitor
<code>htop</code>	Interactive process monitor
<code>pgrep</code> <code>NAME</code>	Find process
<code>kill PID</code>	Terminate process
<code>kill -9</code> <code>PID</code>	Force terminate
<code>pkill</code> <code>NAME</code>	Kill by name

Examples:

```
ps aux
pgrep python
kill 1234
```

Use `kill -9` only when normal termination doesn't work.

12. Background and Foreground Jobs

Run a command in background:

```
python app.py &
```

Show jobs:

```
jobs
```


Bring job to foreground:

`fg`

Suspend a running program:

`Ctrl + Z`

Resume in background:

`bg`

13. Environment Variables

View variables:

`env`
`printenv`

Show one variable:

`echo $PATH`

Create variable:

`export API_KEY="value"`

Remove variable:

`unset API_KEY`

Common variables:

`PATH`
`HOME`
`USER`
`SHELL`
`PWD`

14. PATH

Check PATH:

```
echo $PATH
```

Temporarily add directory:

```
export PATH="$PATH:/my/program"
```

Linux searches directories in `$PATH` when you run commands.

15. Command Location

Find where a command comes from:

```
which python
```

Better diagnostic:

```
type python
```

Find executable:

```
whereis python
```

16. Command History

```
history
```

Search history:

```
Ctrl + R
```

Run previous command:

!!

Run command number:

!123

Clear history:

history -c

17. Terminal Management

Command	Description
<code>clear</code>	Clear terminal
<code>reset</code>	Reset terminal
<code>echo</code>	Print text
<code>printf</code>	Formatted output
<code>date</code>	Current date/time
<code>whoami</code>	Current user
<code>hostname</code>	Computer hostname

18. Package Management — Ubuntu

Ubuntu uses `apt`.

Update package information:

`sudo apt update`

Upgrade packages:

`sudo apt upgrade`

Install:

`sudo apt install PACKAGE`

Remove:

`sudo apt remove PACKAGE`

Search:

`apt search PACKAGE`

Show package information:

`apt show PACKAGE`

Remove unused packages:

`sudo apt autoremove`

Examples:

`sudo apt install tree`

`sudo apt install git`

`sudo apt install curl`

19. Archives

tar

Create archive:

`tar -cf archive.tar project/`

Extract:

`tar -xf archive.tar`

Create gzip archive:

```
tar -czf project.tar.gz project/
```

Extract gzip:

```
tar -xzf project.tar.gz
```

List contents:

```
tar -tf archive.tar
```

20. Compression

gzip file.txt

gunzip file.txt.gz

For ZIP:

zip archive.zip file.txt

unzip archive.zip

Install ZIP tools if necessary:

```
sudo apt install zip unzip
```

21. Disk Management

Check filesystem usage:

```
df -h
```

Check directory size:

```
du -sh project/
```

Check individual items:

`du -sh *`

List block devices:

`lsblk`

22. System Information

`uname -a`

Kernel/system information.

`hostnamectl`

System and hostname information.

`lscpu`

CPU information.

`free -h`

RAM usage.

`lsmem`

Memory information.

`lsblk`

Storage devices.

23. Operating System Information

`cat /etc/os-release`

Shows Linux distribution information.

Example:

Ubuntu

Kernel version:

uname -r

24. Services — systemd

Check service:

`systemctl status SERVICE`

Start:

`sudo systemctl start SERVICE`

Stop:

`sudo systemctl stop SERVICE`

Restart:

`sudo systemctl restart SERVICE`

Enable at boot:

`sudo systemctl enable SERVICE`

Disable at boot:

`sudo systemctl disable SERVICE`

Examples:

```
sudo systemctl status ssh  
sudo systemctl restart docker
```

25. Logs

System logs:

```
journalctl
```

Follow logs:

```
journalctl -f
```

Service logs:

```
journalctl -u SERVICE
```

Example:

```
journalctl -u ssh
```

Recent logs:

```
journalctl -n 50
```

26. Networking

Show IP addresses:

```
ip addr
```

Short version:

```
ip a
```

Show routes:

ip route

Test connectivity:

ping google.com

DNS lookup:

nslookup google.com

or:

dig google.com

Check open/listening ports:

ss -tulpn

27. curl

curl transfers data over protocols such as HTTP/HTTPS.

GET request:

curl https://example.com

Download file:

curl -O https://example.com/file.zip

Follow redirects:

curl -L URL

API request:

curl -X GET https://api.example.com/users

28. wget

Download files:

wget URL

Example:

```
wget https://example.com/file.zip
```

29. SSH

SSH provides secure remote shell access.

Connect:

```
ssh user@server
```

Example:

```
ssh ubuntu@192.168.1.10
```

Specify key:

```
ssh -i key.pem user@server
```

Copy file to server:

```
scp file.txt user@server:/home/user/
```

Copy from server:

```
scp user@server:/home/user/file.txt .
```

30. SSH Keys

Generate key:

ssh-keygen

Copy public key:

ssh-copy-id user@server

Typical files:

~/.ssh/id_ed25519

~/.ssh/id_ed25519.pub

Never share your private key.

31. Git

Check Git:

git --version

Initialize repository:

git init

Clone:

git clone URL

Check status:

git status

Stage:

git add .

Commit:

git commit -m "message"

Push:

`git push`

Pull:

`git pull`

Branches:

`git branch`

`git switch -c feature`

`git switch main`

32. Symbolic Links

Create symlink:

`ln -s TARGET LINK`

Example:

`ln -s /var/www/project ~/project`

View links:

`ls -l`

33. Redirection

Output to file:

`command > file.txt`

Append:

`command >> file.txt`

Input:

```
command < file.txt
```

Error redirection:

```
command 2> error.log
```

Output + error:

```
command > output.log 2>&1
```

34. Pipes

A pipe sends one command's output to another command.

```
command1 | command2
```

Example:

```
ps aux | grep python
```

Another example:

```
ls -la | less
```

35. Useful Text Tools

sort

```
sort file.txt
```

Sort lines.

uniq

```
sort file.txt | uniq
```

Remove adjacent duplicates.

cut

```
cut -d: -f1 /etc/passwd
```

Extract fields.

awk

```
awk '{print $1}' file.txt
```

Process structured text.

sed

```
sed 's/old/new/g' file.txt
```

Replace text.

36. xargs

Pass output as command arguments.

```
find . -name "*.log" | xargs rm
```

Use carefully when deleting files.

37. Shell Scripting

Create:

```
touch script.sh
```

Make executable:

```
chmod +x script.sh
```

Run:

```
./script.sh
```

Basic script:

```
#!/bin/bash
```

```
echo "Hello Linux"
```

Variables:

```
name="Sheharyar"  
echo "$name"
```

Arguments:

```
echo "$1"
```

Condition:

```
if [ "$1" = "hello" ]; then  
    echo "Hello"  
fi
```

Loop:

```
for file in *.txt; do  
    echo "$file"  
done
```

38. Bash Operators

Logical AND:

```
command1 && command2
```

Run second only if first succeeds.

Logical OR:

```
command1 || command2
```

Run second if first fails.

Sequential:

```
command1 ; command2
```

39. File Descriptors

Linux commonly uses:

0 → stdin

1 → stdout

2 → stderr

Example:

```
command > output.txt
```

Redirect stdout.

```
command 2> error.txt
```

Redirect stderr.

```
command &> output.txt
```

Redirect stdout and stderr in Bash.

40. Process Signals

Common signals:

SIGTERM → 15 → graceful termination

SIGKILL → 9 → force termination

SIGINT → 2 → interrupt

SIGHUP → 1 → hangup

Examples:

kill PID

kill -9 PID

Prefer normal `kill`/SIGTERM before `kill -9`.

41. Jobs

List jobs:

jobs

Background:

command &

Suspend:

Ctrl + Z

Resume background:

bg

Foreground:

fg

42. Cron Jobs

Edit scheduled tasks:

```
crontab -e
```

View:

```
crontab -l
```

Cron example:

```
0 2 * * * /home/user/backup.sh
```

Runs the script every day at 2:00 AM.

43. Time and Scheduling

date

Show current date/time.

```
sleep 5
```

Wait 5 seconds.

44. Users and Permissions Files

Important files:

/etc/passwd → user accounts

/etc/shadow → password hashes

/etc/group → groups

/etc/sudoers → sudo configuration

Don't casually edit `/etc/sudoers`.

Use:

sudo visudo

when modifying sudo configuration.

45. Important System Directories

/	→ filesystem root
/home	→ user home directories
/root	→ root's home
/etc	→ configuration
/var	→ variable data/logs
/tmp	→ temporary files
/usr	→ applications/libraries
/opt	→ optional software
/dev	→ devices
/proc	→ process/kernel information
/sys	→ kernel/device information
/mnt	→ temporary mounts
/media	→ removable media

46. Mounting

Show mounted filesystems:

mount

Show filesystem usage:

df -h

Mount a filesystem:

sudo mount DEVICE DIRECTORY

Unmount:

sudo umount DIRECTORY

47. Environment and Shell

Current shell:

```
echo $SHELL
```

Current home:

```
echo $HOME
```

Current user:

```
echo $USER
```

Current directory:

```
echo $PWD
```

Reload Bash configuration:

```
source ~/.bashrc
```

48. Aliases

Create temporary alias:

```
alias ll='ls -la'
```

Remove:

```
unalias ll
```

Permanent aliases can be placed in:

```
~/.bashrc
```

49. man

Linux manual pages:

man COMMAND

Examples:

man ls

man cp

man chmod

Search manuals:

man -k keyword

Exit:

q

50. Help

Command help:

command --help

Example:

cp --help

For Bash built-ins:

help cd

51. Terminal Shortcuts

Shortcut	Description
<code>Ctrl + C</code>	Stop current command
<code>Ctrl + Z</code>	Suspend process
<code>Ctrl + D</code>	Exit shell / EOF
<code>Ctrl + L</code>	Clear terminal
<code>Ctrl + R</code>	Search command history
<code>Ctrl + A</code>	Beginning of line
<code>Ctrl + E</code>	End of line
<code>Ctrl + U</code>	Delete before cursor
<code>Ctrl + K</code>	Delete after cursor
<code>Tab</code>	Autocomplete

52. Docker

Check:

```
docker --version
```

List containers:

```
docker ps
docker ps -a
```

List images:

docker images

Pull image:

docker pull IMAGE

Run container:

docker run IMAGE

Build:

docker build -t myapp .

Stop:

docker stop CONTAINER

Remove:

docker rm CONTAINER

View logs:

docker logs CONTAINER

Execute command inside container:

docker exec -it CONTAINER bash

53. Python Environment

Check Python:

python3 --version

Create virtual environment:

python3 -m venv .venv

Activate:

```
source .venv/bin/activate
```

Deactivate:

```
deactivate
```

Install:

```
pip install PACKAGE
```

Requirements:

```
pip install -r requirements.txt
```

54. Node.js

Check:

```
node --version  
npm --version
```

Install packages:

```
npm install
```

Run project:

```
npm run dev
```

55. AI/ML Useful Commands

Check NVIDIA GPU:

```
nvidia-smi
```


Check CUDA compiler:

```
nvcc --version
```

Check GPU processes:

```
nvidia-smi
```

Check Python:

```
python3 --version
```

Check installed Python packages:

```
pip list
```

This is especially useful when working with:

PyTorch

TensorFlow

CUDA

NVIDIA GPUs

Docker

Python

56. Logs for AI/Production Debugging

Follow application logs:

```
tail -f app.log
```

Search errors:

```
grep -i "error" app.log
```

Search recursively:

```
grep -ri "error" .
```

Check system logs:

```
journalctl -xe
```

Check a service:

```
journalctl -u SERVICE
```

57. Network Debugging

Check IP:

```
ip a
```

Check routes:

```
ip route
```

Test host:

```
ping example.com
```

Test HTTP:

```
curl -I https://example.com
```

DNS:

```
dig example.com
```

Listening ports:

```
ss -tulpn
```

Test specific port:

```
nc -zv HOST PORT
```

58. Common Engineering Workflow

```
cd project/  
ls -la  
git status  
git pull
```

```
source .venv/bin/activate
```

```
python3 app.py
```

```
git add .  
git commit -m "update"  
git push
```

For a server:

```
ssh user@server  
cd /app  
git pull  
sudo systemctl restart app  
sudo systemctl status app  
journalctl -u app -f
```

59. Essential Commands to Memorize

Navigation

```
pwd  
ls  
cd
```

Files

```
mkdir  
touch  
cp  
mv  
rm
```

Viewing

cat
less
head
tail

Search

find
grep

Permissions

chmod
chown
sudo

Users

whoami
id
groups

Processes

ps
top
htop
kill

Packages

apt

Services

systemctl
journalctl

Networking

ip
ping
curl
ss

Remote servers

ssh
scp

Development

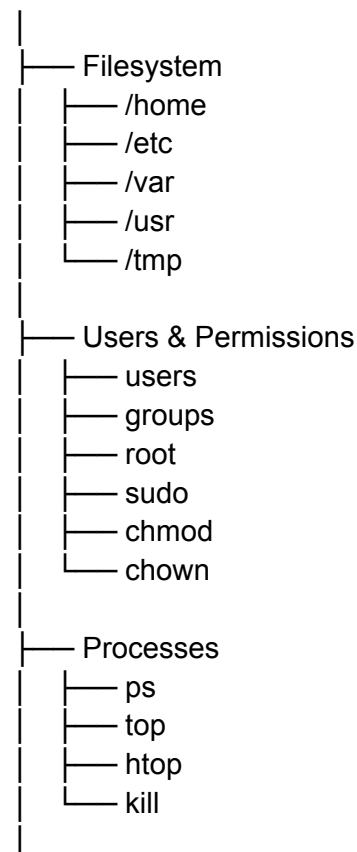
git
python3
pip
node
npm
docker

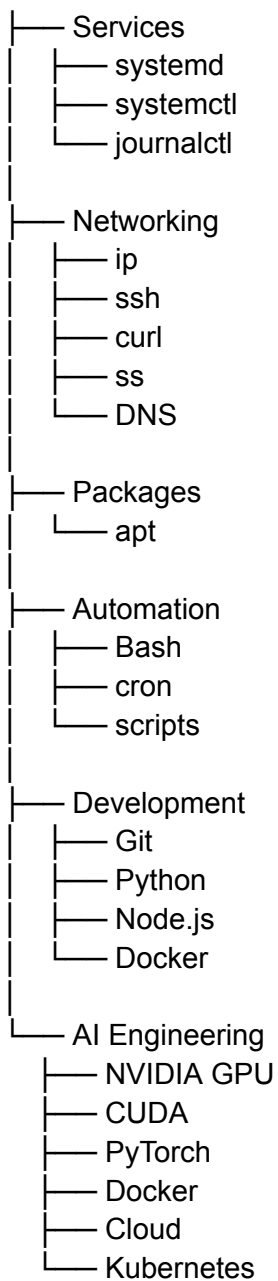
Help

man
--help

60. Linux Engineer Mental Model

Linux





Priority for an AI/Software Engineer

Learn in this order:

1. pwd / ls / cd
2. mkdir / touch / cp / mv / rm
3. cat / less / head / tail
4. find / grep
5. Absolute & relative paths
6. Users / groups / root
7. sudo
8. chmod / chown

9. Processes / ps / top / kill
10. Environment variables
11. apt
12. systemctl / journalctl
13. Networking / SSH
14. Bash scripting
15. Git
16. Docker
17. Cloud Linux servers
18. Kubernetes
19. NVIDIA / CUDA
20. AI/ML deployment

Core principle: Don't try to memorize every command. Learn the command families and use `man COMMAND` or `COMMAND --help` when you need the exact options.

Linux Engineer Command Reference

A concise practical reference covering the most important Linux commands for software, AI/ML, DevOps, cloud, and backend engineers.

1. Navigation

Command	Description
<code>pwd</code>	Show current directory
<code>ls</code>	List files and directories
<code>ls -l</code>	Detailed listing
<code>ls -la</code>	Detailed listing including hidden files
<code>cd DIR</code>	Change directory
<code>cd ..</code>	Go to parent directory
<code>cd .</code>	Stay in current directory
<code>cd ~</code>	Go to home directory
<code>cd /</code>	Go to filesystem root

```
cd -      Go to previous directory
pwd
ls -la
cd /home/sheharyar
cd ..
cd ~
```

2. Linux Paths

```
/      → filesystem root
~      → current user's home
.      → current directory
..     → parent directory
```

Absolute Path

```
/home/sheharyar/project/main.py
```

Starts from `/`.

Relative Path

```
project/main.py
```

Starts from the current directory.

3. Files and Directories

Command	Description
<code>mkdir DIR</code>	Create directory
<code>mkdir -p a/b/c</code>	Create nested directories
<code>touch FILE</code>	Create empty file

<code>cp FILE DEST</code>	Copy file
<code>cp -r DIR DEST</code>	Copy directory
<code>mv OLD NEW</code>	Rename/move
<code>rm FILE</code>	Delete file
<code>rm -r DIR</code>	Delete directory recursively
<code>rmdir DIR</code>	Delete empty directory
<code>rm -i FILE</code>	Ask before deleting
<code>rm -rf DIR</code>	Force recursive deletion

Examples:

```
mkdir project
touch main.py
cp main.py backup.py
mv backup.py backup_main.py
rm backup_main.py
```

4. Viewing Files

Command	Description
<code>cat FILE</code>	Display entire file
<code>less FILE</code>	Read file page by page
<code>head FILE</code>	Show beginning
<code>head -n 20 FILE</code>	Show first 20 lines
<code>tail FILE</code>	Show end
<code>tail -n 20 FILE</code>	Show last 20 lines

`tail -f FILE` Follow a changing file

`nl FILE` Show line numbers

Examples:

```
cat README.md
less README.md
head -n 10 app.log
tail -f app.log
```

Press `q` to exit `less`.

5. Searching Files and Text

find

Search for files/directories.

```
find . -name "main.py"
```

Search by type:

```
find . -type f
find . -type d
```

Search by extension:

```
find . -name "*.py"
```

grep

Search text inside files.

```
grep "error" app.log
```

Useful options:

```
grep -i "error" app.log
grep -r "TODO" .
grep -n "error" app.log
```

-i → ignore case
-r → recursive
-n → show line number

6. File Information

Command	Description
<code>file</code> <code>FILE</code>	Identify file type
<code>stat</code> <code>FILE</code>	Detailed file metadata
<code>wc FILE</code>	Count lines/words/bytes
<code>du -h</code>	Disk usage
<code>df -h</code>	Filesystem disk space

Examples:

```
file model.pkl
stat model.pkl
du -sh project/
df -h
```

7. Permissions

Linux permissions are commonly represented as:

r → read
w → write
x → execute

Example:

`-rwxr-xr--`

Change permissions:

```
chmod 755 script.sh
chmod +x script.sh
chmod -x script.sh
```

Common permissions:

```
755 → rwxr-xr-x
644 → rw-r--r--
600 → rw-----
```

8. Ownership

Check ownership:

```
ls -l
```

Change owner:

```
sudo chown user file.txt
```

Change owner and group:

```
sudo chown user:group file.txt
```

Change directory recursively:

```
sudo chown -R user:group project/
```

Change group:

```
sudo chgrp group file.txt
```

9. Users

Command	Description
<code>whoami</code>	Current username
<code>id</code>	UID, GID and groups
<code>groups</code>	Current user's groups
<code>who</code>	Logged-in users
<code>w</code>	Logged-in users + activity
<code>adduser USER</code>	Create user
<code>deluser USER</code>	Delete user
<code>passwd USER</code>	Change password
<code>usermod</code>	Modify user

Examples:

```
whoami
id
groups
sudo adduser ali
sudo passwd ali
```

10. Root and Sudo

`root` is the Linux superuser.

`sudo` command

Run a command with elevated privileges.

```
sudo apt update
sudo systemctl restart ssh
sudo whoami
```

Become root temporarily:

```
sudo -i
```

Leave root:

```
exit
```

Switch user:

```
su - username
```

11. Processes

A process is a running program.

Command	Description
<code>ps</code>	Show processes
<code>ps aux</code>	Detailed process list
<code>top</code>	Real-time process monitor
<code>htop</code>	Interactive process monitor
<code>pgrep</code> <code>NAME</code>	Find process
<code>kill PID</code>	Terminate process
<code>kill -9</code> <code>PID</code>	Force terminate
<code>pkill</code> <code>NAME</code>	Kill by name

Examples:

```
ps aux  
pgrep python  
kill 1234
```

Use `kill -9` only when normal termination doesn't work.

12. Background and Foreground Jobs

Run a command in background:

```
python app.py &
```

Show jobs:

```
jobs
```

Bring job to foreground:

```
fg
```

Suspend a running program:

```
Ctrl + Z
```

Resume in background:

```
bg
```

13. Environment Variables

View variables:

```
env  
printenv
```

Show one variable:

```
echo $PATH
```

Create variable:

```
export API_KEY="value"
```

Remove variable:

```
unset API_KEY
```

Common variables:

```
PATH  
HOME  
USER  
SHELL  
PWD
```

14. PATH

Check PATH:

```
echo $PATH
```

Temporarily add directory:

```
export PATH="$PATH:/my/program"
```

Linux searches directories in `$PATH` when you run commands.

15. Command Location

Find where a command comes from:

```
which python
```

Better diagnostic:

```
type python
```

Find executable:

whereis python

16. Command History

history

Search history:

Ctrl + R

Run previous command:

!!

Run command number:

!123

Clear history:

history -c

17. Terminal Management

Command	Description
<code>clear</code>	Clear terminal
<code>reset</code>	Reset terminal
<code>echo</code>	Print text
<code>printf</code>	Formatted output
<code>date</code>	Current date/time
<code>whoami</code>	Current user

hostname Computer
hostname

18. Package Management — Ubuntu

Ubuntu uses `apt`.

Update package information:

```
sudo apt update
```

Upgrade packages:

```
sudo apt upgrade
```

Install:

```
sudo apt install PACKAGE
```

Remove:

```
sudo apt remove PACKAGE
```

Search:

```
apt search PACKAGE
```

Show package information:

```
apt show PACKAGE
```

Remove unused packages:

```
sudo apt autoremove
```

Examples:

```
sudo apt install tree
```

```
sudo apt install git
```

```
sudo apt install curl
```

19. Archives

tar

Create archive:

```
tar -cf archive.tar project/
```

Extract:

```
tar -xf archive.tar
```

Create gzip archive:

```
tar -czf project.tar.gz project/
```

Extract gzip:

```
tar -xzf project.tar.gz
```

List contents:

```
tar -tf archive.tar
```

20. Compression

```
gzip file.txt  
gunzip file.txt.gz
```

For ZIP:

```
zip archive.zip file.txt  
unzip archive.zip
```

Install ZIP tools if necessary:

sudo apt install zip unzip

21. Disk Management

Check filesystem usage:

df -h

Check directory size:

du -sh project/

Check individual items:

du -sh *

List block devices:

lsblk

22. System Information

uname -a

Kernel/system information.

hostnamectl

System and hostname information.

lscpu

CPU information.

free -h

RAM usage.

lsmem

Memory information.

lsblk

Storage devices.

23. Operating System Information

`cat /etc/os-release`

Shows Linux distribution information.

Example:

Ubuntu

Kernel version:

`uname -r`

24. Services — systemd

Check service:

`systemctl status SERVICE`

Start:

`sudo systemctl start SERVICE`

Stop:

`sudo systemctl stop SERVICE`

Restart:

```
sudo systemctl restart SERVICE
```

Enable at boot:

```
sudo systemctl enable SERVICE
```

Disable at boot:

```
sudo systemctl disable SERVICE
```

Examples:

```
sudo systemctl status ssh  
sudo systemctl restart docker
```

25. Logs

System logs:

```
journalctl
```

Follow logs:

```
journalctl -f
```

Service logs:

```
journalctl -u SERVICE
```

Example:

```
journalctl -u ssh
```

Recent logs:

```
journalctl -n 50
```

26. Networking

Show IP addresses:

```
ip addr
```

Short version:

```
ip a
```

Show routes:

```
ip route
```

Test connectivity:

```
ping google.com
```

DNS lookup:

```
nslookup google.com
```

or:

```
dig google.com
```

Check open/listening ports:

```
ss -tulpn
```

27. curl

`curl` transfers data over protocols such as HTTP/HTTPS.

GET request:

```
curl https://example.com
```

Download file:

```
curl -O https://example.com/file.zip
```

Follow redirects:

```
curl -L URL
```

API request:

```
curl -X GET https://api.example.com/users
```

28. wget

Download files:

```
wget URL
```

Example:

```
wget https://example.com/file.zip
```

29. SSH

SSH provides secure remote shell access.

Connect:

```
ssh user@server
```

Example:

```
ssh ubuntu@192.168.1.10
```

Specify key:

```
ssh -i key.pem user@server
```

Copy file to server:


```
scp file.txt user@server:/home/user/
```

Copy from server:

```
scp user@server:/home/user/file.txt .
```

30. SSH Keys

Generate key:

```
ssh-keygen
```

Copy public key:

```
ssh-copy-id user@server
```

Typical files:

```
~/.ssh/id_ed25519
```

```
~/.ssh/id_ed25519.pub
```

Never share your private key.

31. Git

Check Git:

```
git --version
```

Initialize repository:

```
git init
```

Clone:

```
git clone URL
```

Check status:

```
git status
```

Stage:

```
git add .
```

Commit:

```
git commit -m "message"
```

Push:

```
git push
```

Pull:

```
git pull
```

Branches:

```
git branch  
git switch -c feature  
git switch main
```

32. Symbolic Links

Create symlink:

```
ln -s TARGET LINK
```

Example:

```
ln -s /var/www/project ~/project
```

View links:

```
ls -l
```

33. Redirection

Output to file:

```
command > file.txt
```

Append:

```
command >> file.txt
```

Input:

```
command < file.txt
```

Error redirection:

```
command 2> error.log
```

Output + error:

```
command > output.log 2>&1
```

34. Pipes

A pipe sends one command's output to another command.

```
command1 | command2
```

Example:

```
ps aux | grep python
```

Another example:

```
ls -la | less
```

35. Useful Text Tools

sort

`sort file.txt`

Sort lines.

uniq

`sort file.txt | uniq`

Remove adjacent duplicates.

cut

`cut -d: -f1 /etc/passwd`

Extract fields.

awk

`awk '{print $1}' file.txt`

Process structured text.

sed

`sed 's/old/new/g' file.txt`

Replace text.

36. xargs

Pass output as command arguments.

`find . -name "*.log" | xargs rm`

Use carefully when deleting files.

37. Shell Scripting

Create:

```
touch script.sh
```

Make executable:

```
chmod +x script.sh
```

Run:

```
./script.sh
```

Basic script:

```
#!/bin/bash
```

```
echo "Hello Linux"
```

Variables:

```
name="Sheharyar"  
echo "$name"
```

Arguments:

```
echo "$1"
```

Condition:

```
if [ "$1" = "hello" ]; then  
    echo "Hello"  
fi
```

Loop:

```
for file in *.txt; do
    echo "$file"
done
```

38. Bash Operators

Logical AND:

```
command1 && command2
```

Run second only if first succeeds.

Logical OR:

```
command1 || command2
```

Run second if first fails.

Sequential:

```
command1 ; command2
```

39. File Descriptors

Linux commonly uses:

```
0 → stdin
1 → stdout
2 → stderr
```

Example:

```
command > output.txt
```

Redirect stdout.

```
command 2> error.txt
```

Redirect stderr.

command &> output.txt

Redirect stdout and stderr in Bash.

40. Process Signals

Common signals:

SIGTERM → 15 → graceful termination

SIGKILL → 9 → force termination

SIGINT → 2 → interrupt

SIGHUP → 1 → hangup

Examples:

kill PID

kill -9 PID

Prefer normal `kill`/SIGTERM before `kill -9`.

41. Jobs

List jobs:

jobs

Background:

command &

Suspend:

Ctrl + Z

Resume background:

bg

Foreground:

fg

42. Cron Jobs

Edit scheduled tasks:

```
crontab -e
```

View:

```
crontab -l
```

Cron example:

```
0 2 * * * /home/user/backup.sh
```

Runs the script every day at 2:00 AM.

43. Time and Scheduling

date

Show current date/time.

```
sleep 5
```

Wait 5 seconds.

44. Users and Permissions Files

Important files:

/etc/passwd → user accounts
/etc/shadow → password hashes
/etc/group → groups
/etc/sudoers → sudo configuration

Don't casually edit `/etc/sudoers`.

Use:

`sudo visudo`

when modifying sudo configuration.

45. Important System Directories

/ → filesystem root
/home → user home directories
/root → root's home
/etc → configuration
/var → variable data/logs
/tmp → temporary files
/usr → applications/libraries
/opt → optional software
/dev → devices
/proc → process/kernel information
/sys → kernel/device information
/mnt → temporary mounts
/media → removable media

46. Mounting

Show mounted filesystems:

`mount`

Show filesystem usage:

`df -h`

Mount a filesystem:

`sudo mount DEVICE DIRECTORY`

Unmount:

`sudo umount DIRECTORY`

47. Environment and Shell

Current shell:

`echo $SHELL`

Current home:

`echo $HOME`

Current user:

`echo $USER`

Current directory:

`echo $PWD`

Reload Bash configuration:

`source ~/.bashrc`

48. Aliases

Create temporary alias:

`alias ll='ls -la'`

Remove:

`unalias ll`

Permanent aliases can be placed in:

`~/.bashrc`

49. **man**

Linux manual pages:

`man COMMAND`

Examples:

`man ls`

`man cp`

`man chmod`

Search manuals:

`man -k keyword`

Exit:

`q`

50. **Help**

Command help:

`command --help`

Example:

cp --help

For Bash built-ins:

help cd

51. Terminal Shortcuts

Shortcut	Description
Ctrl + C	Stop current command
Ctrl + Z	Suspend process
Ctrl + D	Exit shell / EOF
Ctrl + L	Clear terminal
Ctrl + R	Search command history
Ctrl + A	Beginning of line
Ctrl + E	End of line
Ctrl + U	Delete before cursor
Ctrl + K	Delete after cursor
Tab	Autocomplete

52. Docker

Check:

`docker --version`

List containers:

`docker ps`

`docker ps -a`

List images:

`docker images`

Pull image:

`docker pull IMAGE`

Run container:

`docker run IMAGE`

Build:

`docker build -t myapp .`

Stop:

`docker stop CONTAINER`

Remove:

`docker rm CONTAINER`

View logs:

`docker logs CONTAINER`

Execute command inside container:

`docker exec -it CONTAINER bash`

53. Python Environment

Check Python:

```
python3 --version
```

Create virtual environment:

```
python3 -m venv .venv
```

Activate:

```
source .venv/bin/activate
```

Deactivate:

```
deactivate
```

Install:

```
pip install PACKAGE
```

Requirements:

```
pip install -r requirements.txt
```

54. Node.js

Check:

```
node --version  
npm --version
```

Install packages:

```
npm install
```

Run project:

npm run dev

55. AI/ML Useful Commands

Check NVIDIA GPU:

`nvidia-smi`

Check CUDA compiler:

`nvcc --version`

Check GPU processes:

`nvidia-smi`

Check Python:

`python3 --version`

Check installed Python packages:

`pip list`

This is especially useful when working with:

PyTorch
TensorFlow
CUDA
NVIDIA GPUs
Docker
Python

56. Logs for AI/Production Debugging

Follow application logs:

`tail -f app.log`

Search errors:

```
grep -i "error" app.log
```

Search recursively:

```
grep -ri "error" .
```

Check system logs:

```
journalctl -xe
```

Check a service:

```
journalctl -u SERVICE
```

57. Network Debugging

Check IP:

```
ip a
```

Check routes:

```
ip route
```

Test host:

```
ping example.com
```

Test HTTP:

```
curl -I https://example.com
```

DNS:

```
dig example.com
```


Listening ports:

```
ss -tulpn
```

Test specific port:

```
nc -zv HOST PORT
```

58. Common Engineering Workflow

```
cd project/
```

```
ls -la
```

```
git status
```

```
git pull
```

```
source .venv/bin/activate
```

```
python3 app.py
```

```
git add .
```

```
git commit -m "update"
```

```
git push
```

For a server:

```
ssh user@server
```

```
cd /app
```

```
git pull
```

```
sudo systemctl restart app
```

```
sudo systemctl status app
```

```
journalctl -u app -f
```

59. Essential Commands to Memorize

Navigation

```
pwd
```

```
ls
```

```
cd
```

Files

mkdir
touch
cp
mv
rm

Viewing

cat
less
head
tail

Search

find
grep

Permissions

chmod
chown
sudo

Users

whoami
id
groups

Processes

ps
top
htop
kill

Packages

apt

Services

systemctl
journalctl

Networking

ip
ping
curl
ss

Remote servers

ssh
scp

Development

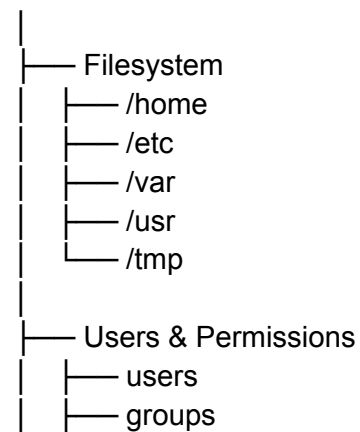
git
python3
pip
node
npm
docker

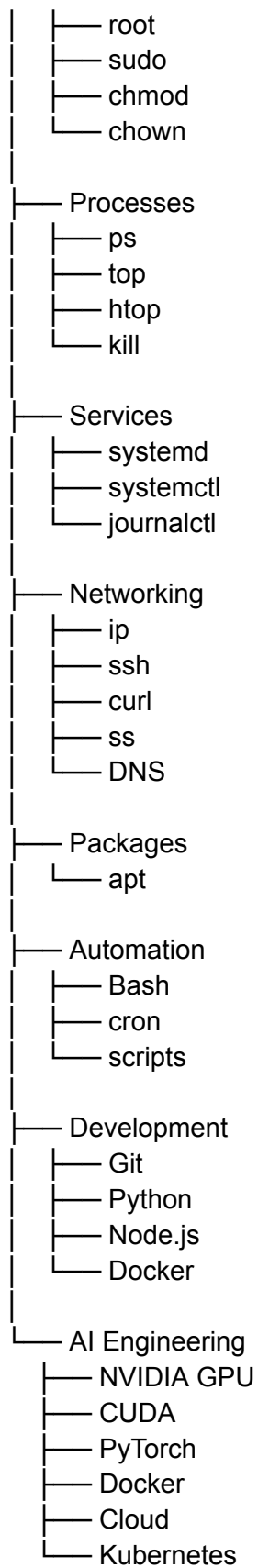
Help

man
--help

60. Linux Engineer Mental Model

Linux





Priority for an AI/Software Engineer

Learn in this order:

1. pwd / ls / cd
2. mkdir / touch / cp / mv / rm
3. cat / less / head / tail
4. find / grep
5. Absolute & relative paths
6. Users / groups / root
7. sudo
8. chmod / chown
9. Processes / ps / top / kill
10. Environment variables
11. apt
12. systemctl / journalctl
13. Networking / SSH
14. Bash scripting
15. Git
16. Docker
17. Cloud Linux servers
18. Kubernetes
19. NVIDIA / CUDA
20. AI/ML deployment

Core principle: Don't try to memorize every command. Learn the command families and use `man COMMAND` or `COMMAND --help` when you need the exact options.